

CSE 203: Data Structures

Lecture 4: Elementary Data Structures- Queue

Queues

FIFO: First in, First Out

Restricted form of list: Insert at one end, remove from the other.

Notation:

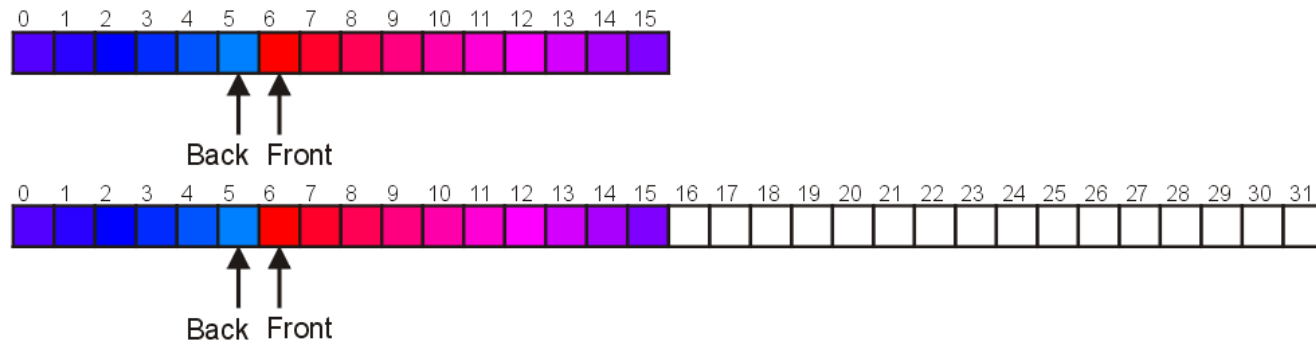
- Insert: Enqueue
- Delete: Dequeue
- First element: Front
- Last element: Rear



Increasing Capacity

Unfortunately, if we choose to increase the capacity, this becomes slightly more complex

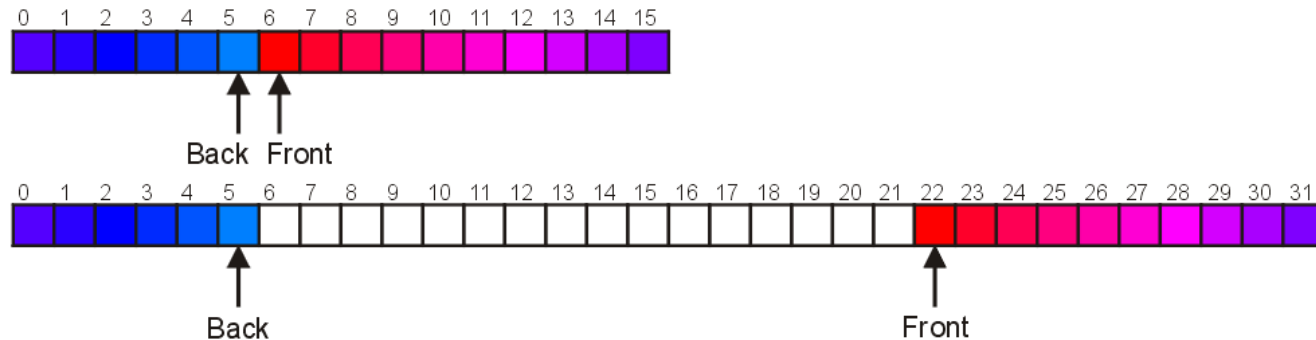
- A direct copy does not work:



Increasing Capacity

There are two solutions:

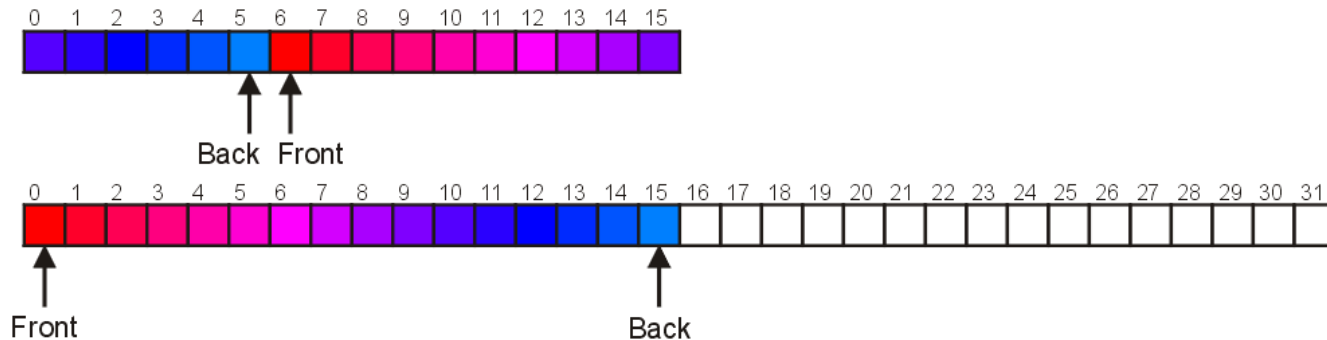
- Move those beyond the front to the end of the array
- The next push would then occur in position 6



Increasing Capacity

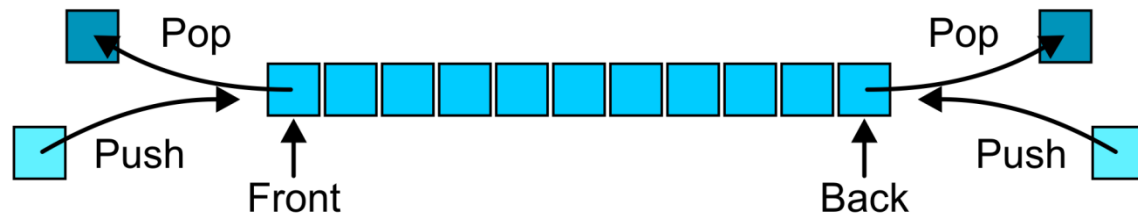
An alternate solution is normalization:

- Map the front back at position 0
- The next push would then occur in position 16



Abstract Deque

An Abstract Deque (Deque ADT) is an abstract data structure that Allows insertions at and removal from both the front and back of the deque



Abstract Deque

The operations will be called

front	back
push_front	push_back
pop_front	pop_back

There are four errors associated with this abstract data type:

- It is an undefined operation to access or pop from an empty deque

Applications

Useful as a general-purpose tool:

- Can be used as either a queue or a stack

Implementations

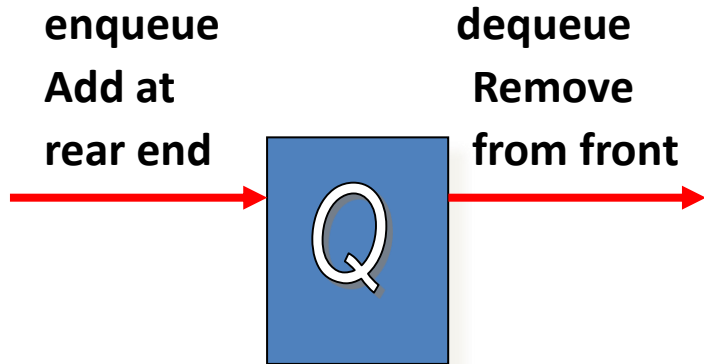
The implementations are clear:

- We must use either a doubly linked list or a circular array

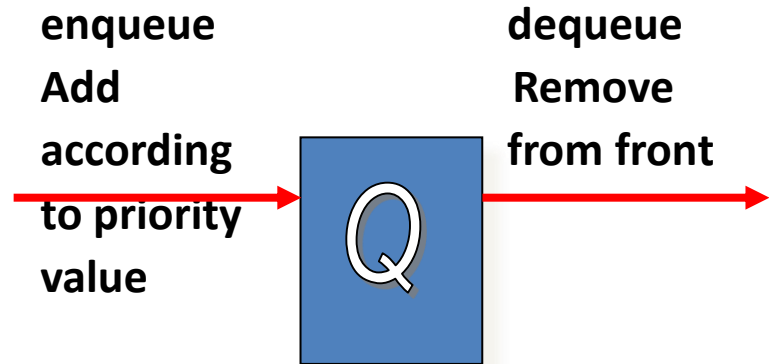
Priority Queues

A queue that is ordered according to some priority value

Standard Queue



Priority Queue



(MAX/MIN)-Priority Queues

- The heap data structure is incredibly useful for implementing *priority queues*
 - A data structure for maintaining a set S of elements, each with an associated value or *key*
- Applications:
 - we can use max-priority queues to schedule jobs on a shared computer.
 - The max-priority queue keeps track of the jobs to be performed and their relative priorities.
 - When a job is finished or interrupted, the scheduler selects the highest-priority job from among those pending.
 - The scheduler can add a new job to the queue at any time

Priority Queue Operations

$\text{Insert}(S, x)$ – Inserts element x into set S , according to its priority

$\text{Maximum}(S)$ – Returns, but does not remove, the element of S with the largest key

$\text{Extract-Max}(S)$ – Removes and returns the element of S with the largest key

$\text{Increase-Key}(S, x, k)$ – Increases the value of element x 's key to the new value k

□ *How could we implement these operations using a heap?*

Priority Queue Operations

HEAP-MAXIMUM(A)

1 **return** $A[1]$

HEAP-EXTRACT-MAX(A)

1 **if** $\text{heap-size}[A] < 1$

2 **then error** “heap underflow”

3 $\text{max} \leftarrow A[1]$

4 $A[1] \leftarrow A[\text{heap-size}[A]]$

5 $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$

6 MAX-HEAPIFY($A, 1$)

7 **return** max

Priority Queue Operations

HEAP-INCREASE-KEY(A, i, key)

```
1  if  $key < A[i]$ 
2      then error “new key is smaller than current key”
3   $A[i] \leftarrow key$ 
4  while  $i > 1$  and  $A[PARENT(i)] < A[i]$ 
5      do exchange  $A[i] \leftrightarrow A[PARENT(i)]$ 
6       $i \leftarrow PARENT(i)$ 
```

Priority Queue Operations

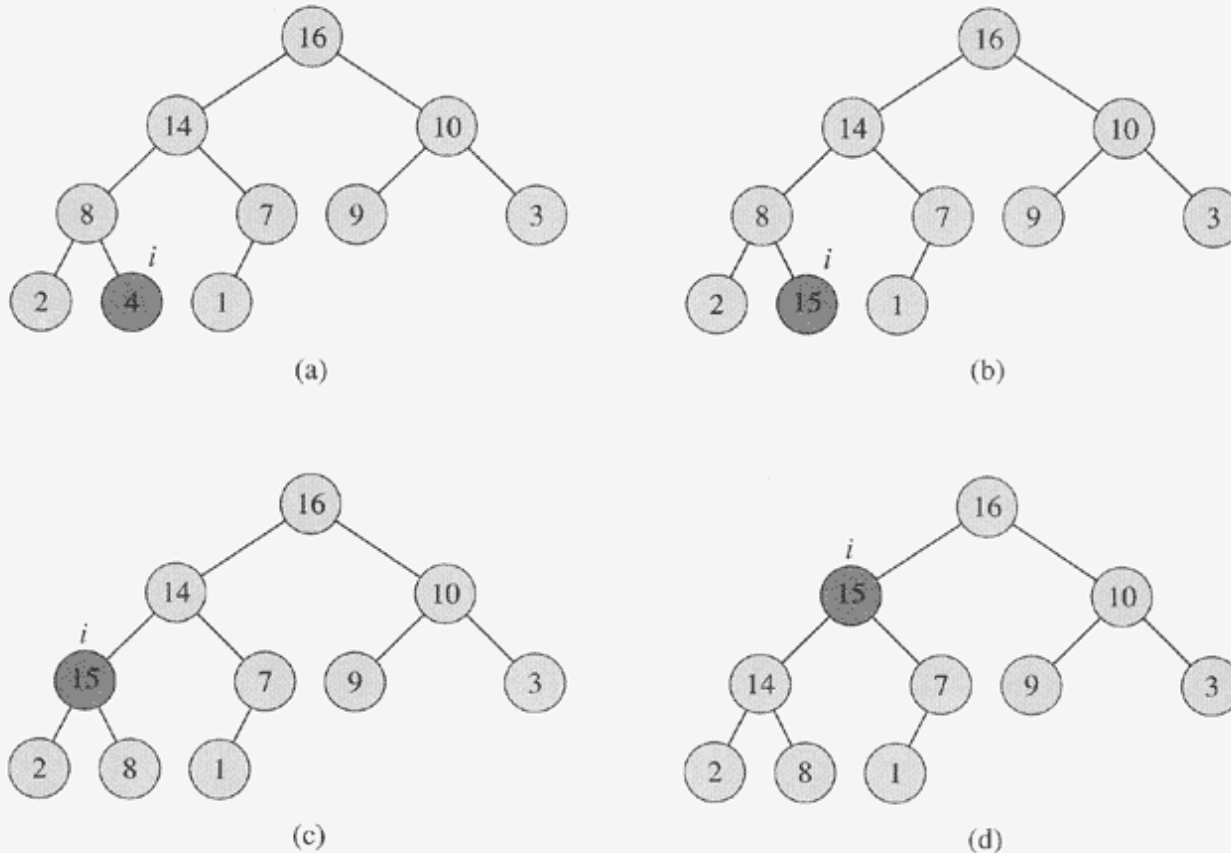


Figure 6.5 The operation of HEAP-INCREASE-KEY. (a) The max-heap of Figure 6.4(a) with a node whose index is i heavily shaded. (b) This node has its key increased to 15. (c) After one iteration of the **while** loop of lines 4–6, the node and its parent have exchanged keys, and the index i moves up to the parent. (d) The max-heap after one more iteration of the **while** loop. At this point, $A[\text{PARENT}(i)] \geq A[i]$. The max-heap property now holds and the procedure terminates.

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $heap-size[A] \leftarrow heap-size[A] + 1$

2 $A[heap-size[A]] \leftarrow -\infty$

3 HEAP-INCREASE-KEY($A, heap-size[A], key$)